Programação Orientada a Objetos

Aula 03 - Atributos e Métodos de Classe e Encapsulamento

Prof. Dr. Eduardo de Olivindo (eduardo.olivindo@ifce.edu.br)

Tecnologia em Análise e Desenvolvimento de Sistemas

IFCE - Campus Itapipoca



Objetivos da aula

- Compreender atributos e métodos de classe
- Criar classes próprias com atributos e métodos da classe
- Compreender a necessidade do uso de encapsulamento
- Criar classes prórprias com atributos e métodos encapsulados

Atributos de Classe

Atributos de classe

Informação compartilhada

- São diferentes dos atributos de instância
- É definido no corpo da classe, fora dos métodos e não usa o prefixo `self`.
- Existe um único valor compartilhado pela classe.

```
class Conta:
  taxa_geral = 0.03

  def __init__(self, titular: str, taxa: float):
    self.titular = titular
    self.taxa = taxa
```

`taxa_geral` pertence à classe; `titular` e `taxa` pertencem a cada instância.

Atributos de classe

Como acessar?

- Use `NomeDaClasse.atributo` para acessar o atributo.

```
print(Conta.taxa_geral) # Existe antes dos objetos existirem

c = Conta("Ana", 0.05)
print(c.taxa)        # atributo da instância
print(Conta.taxa_geral) # atributo da classe
```

- O atributo de classe pode ser consultado mesmo sem criar objetos.
- Cuidado: Lembre que uma alteração afeta todos

Exercício - Atributos de classe

- Crie a classe `Produto` com:
  - um atributo de classe chamado `imposto` com valor inicial `0.10`
  - atributos de instância `nome` e `preco`
  - um método `preco_final(self)` que retorna o preço com o imposto da classe
- Teste a classe criada para entender o funcionamento do atributo da classe.

Métodos de classe

Métodos de classe

- Um método de classe não depende de uma instância específica.
- Deve usar o *decorator* `@classmethod` antes de sua definição.
- O primeiro parâmetro é `cls`, uma referência à própria classe (e não `self`).

```
class Usuario:
    total_usuarios = 0 # atributo da classe

    def __init__(self, nome, idade):
        self.nome = nome         # Dado do objeto
        self.idade = idade       # Dado do objeto
        Usuario.total_usuarios += 1

    @classmethod
    def criar_por_texto(cls, texto):
        nome, idade = texto.split(",")
        return cls(nome, int(idade))
```

Métodos de classe

Como usar?

- Pode ser chamado usando o nome da classe `NomeDaClasse.metodo()`.

```
usuario1 = Usuario.criar_por_texto("Carlos,30")
```

- Métodos da classe podem ser usados antes mesmo de existir um objeto.
- Neste caso foi usado um método de classe como construtor.
- Normalmente são métodos ferramentais da classe, que não precisam de objetos previamente criados.

Exercício - Método da classe

- Para a classe Usuário apresentada no exemplo anterior crie o seguinte método da classe:
  - `mostrar_total_de_usuarios()`: que retorna quantos objetos do tipo usuário foram criados.
- Teste a classe e o método criados.
- Responda: por que `mostrar_total_de_usuarios()` deve ser um método de classe e não um método comum?

Encapsulamento

-

Encapsulamento

- Encapsulamento significa esconder detalhes internos de uma classe.
- A classe expõe uma interface para uso, enquanto mantém o funcionamento interno protegido.
- Isso ajuda a evitar alterações indevidas no estado do objeto.
- Também permite controlar como os dados são lidos e modificados.

Encapsulamento

Isso aqui já é uma espécie de Encapsulamento:

```
class Conta:
    def __init__(self):
        self.saldo = 0

    def sacar(self, valor):
        if valor <= self.saldo:
            self.saldo -= valor
            print("Saque realizado")
        else:
            print("Saldo insuficiente")
```

- O usuário chama `sacar()`, sem precisar conhecer os detalhes internos.
- A classe define as regras e mantém a consistência dos dados.

Encapsulamento

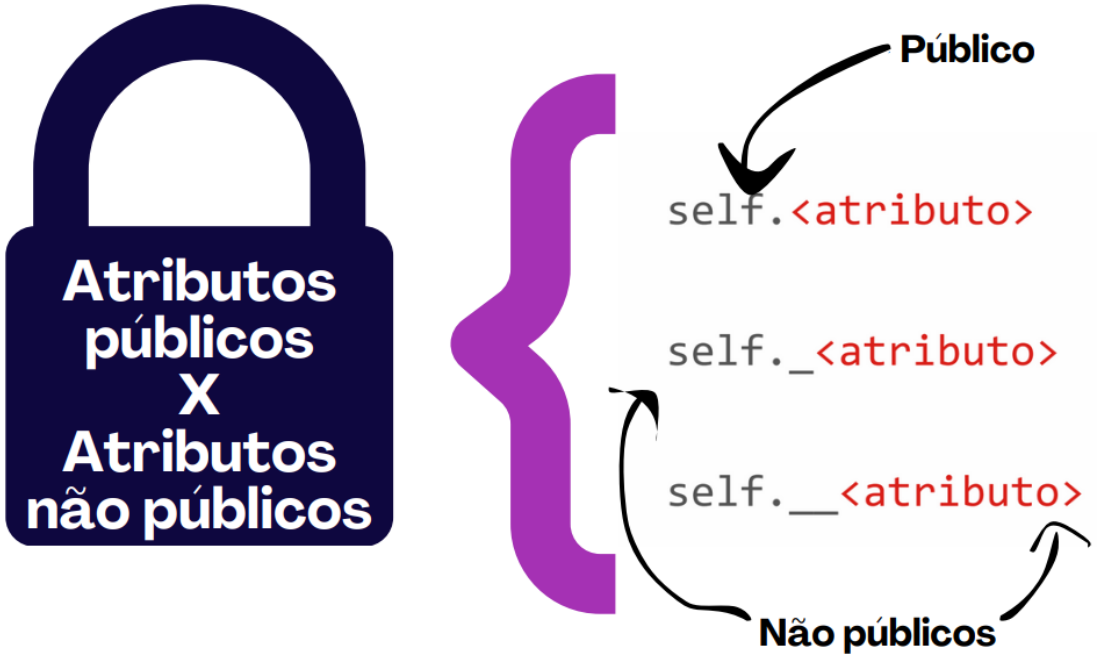
Mas será o suficiente??? e ao usar a classe `conta` se fizermos isso?

```
minha_conta=Conta(100)
minha_conta.saldo=1000000000
minha_conta.sacar(1000000000)
```

- 🤑🤑🤑🤑🤑🤑🤑🤑🤑🤑
- É necessário melhorar o encapsulamento, tornando atributos privados.

Encapsulamento

- Em python encapsulamento é feito com convenções de nomenclatura



- Mas python não trava o acesso a esses atributos

Atributos públicos e privados

- Atributos públicos podem ser acessados diretamente fora da classe.
- Atributos privados são indicados com `__` antes do nome.
- Essa convenção ajuda a restringir o acesso a informações internas.

```
class Carro:
    def __init__(self, modelo, preco):
        self.modelo = modelo      # público
        self.__preco = preco      # privado
```

- `modelo` pode ser acessado diretamente.
- `__preco` deve ser acessado apenas por métodos da própria classe.

Acessando atributos privados -

- Em Python, um atributo privado é renomeado internamente pela linguagem.
- Exemplo: `__preco` vira `_Carro__preco`. Isso é chamado de *name mangling*.

```
class Carro:
    def __init__(self, modelo, preco):
        self.modelo = modelo
        self.__preco = preco

c = Carro("Fiat", 50000)
print(c.modelo)
print(c._Carro__preco)
```

- Isso não impede acesso completo, mas dificulta uso indevido.
- O objetivo é manter a lógica de validação dentro da classe.

Getter e Setter

- Váriáveis não publicas ainda precisam ser acessadas
- `getter` lê o valor de um atributo.
- `setter` altera o valor, geralmente com validação.
- Assim, a classe controla o que pode ser alterado.

```
class Conta:
    def __init__(self, saldo):
        self.__saldo = saldo

    def get_saldo(self):
        return self.__saldo

    def set_saldo(self, valor):
        if valor >= 0:
            self.__saldo = valor
```

Getter e setter em ação

```
c = Conta(100)
print(c.get_saldo())

c.set_saldo(250)
print(c.get_saldo())

c.set_saldo(-10)
```

Saída esperada:

```
100
250
Saldo não pode ser negativo
```

- A lógica de validação está concentrada dentro da classe.

A forma mais Pythonica

- `@property` para o getter | `@atributo.setter` para o setter.

```
class Conta:
    def __init__(self, saldo):
        self.__saldo = saldo

    @property
    def saldo(self):
        return self.__saldo

    @saldo.setter
    def saldo(self, valor):
        if valor >= 0:
            self.__saldo = valor
        else:
            raise ValueError("Saldo não pode ser negativo")
```

A forma mais Pythonica: `@property`

- Agora podemos usar:

```
c = Conta(100)
print(c.saldo)
c.saldo = 200
print(c.saldo)
```

- Acesso igual para qualquer atributo, mas passando pelas regras de acesso.
- O código fica mais legível e mantém a segurança da classe.

Por que usar encapsulamento?

- Protege dados sensíveis.
- Evita alterações sem regras.
- Deixa a classe mais organizada.
- Facilita manutenção e evolução do software.
- Melhora a qualidade do código orientado a objetos.

Exemplo: Classe Pessoa

```python
class Pessoa:
    def __init__(self, nome, idade):
        self.__nome = nome
        self.__idade = idade

    @property
    def nome(self):
        return self.__nome

    @nome.setter
    def nome(self, novo_nome):
        if novo_nome.strip():
            self.__nome = novo_nome
        else:
            raise ValueError("Nome inválido")

    @property
    def idade(self):
        return self.__idade

    @idade.setter
    def idade(self, nova_idade):
        if nova_idade >= 0:
            self.__idade = nova_idade
        else:
            raise ValueError("Idade não pode ser negativa")
```

Exercício - Encapsulamento

- Crie a classe `Livro` com os atributos privados `__titulo` e `__paginas`.
- Crie um getter e um setter para cada atributo.
- No setter de `paginas`, aceite apenas valores numéricos maiores que zero.
- Teste a classe e tente atribuir um valor inválido para verificar a validação.

Encapsulamento - Métodos privados

Encapsulamento - Métodos privados

- Da mesma forma que com atributos, métodos podem ser "escondidos".
- Dois underlines `__` no início do nome do método.
- Utilizado pra métodos que não devem ser acessados por códigos fora da definição da classe.
- Normalmente métodos privados são métodos "ajudantes" da classe

Encapsulamento - Métodos privados

Exemplo:

```
class Recipient:
    def __init__(self, nome: str, email: str):
        self.__nome = nome
        if self.__check_email(email):
            self.__email = email
        else:
            raise ValueError("O email não é valido")

    def __check_email(self, email: str):
        # O endereço de email deve ter mais que 5 caracteres e ter . e @
        return len(email) > 5 and "." in email and "@" in email

r=Recipient("eduardo", "eduardo.olivindo@ifce.edu.br")
r.__check_email("eduardo.olivindo@ifce.edu.br")
```